EDS

Unit 1

Python Fundamentals for Data Science

Introduction to Python for Data Science

- Python is used to **analyze data**, not just write programs
- Engineers deal with:
  - Rainfall data 🌧️
  - Temperature readings 🌡️
  - Load, stress, strain 📐
  - Machine performance 🚜

Real-life linkage

- Civil: Rainfall analysis for flood prediction
- Mechanical: Temperature and vibration data from machines

Python Program

```
print("Welcome to Data Science for Engineers")

rainfall = 120
print("Rainfall today:", rainfall, "mm")


temperature = 75
print("Machine temperature:", temperature, "°C")
```

Data Types – Mutable and Immutable

Basic Data Types

Data Type	Example
int	`10`
float	`36.5`
string	`"Concrete"`
boolean	`True/False`

ext Type:	`str`
Numeric Types:	`int`, `float`, `complex`
Sequence Types:	`list`, `tuple`, `range`
Mapping Type:	`dict`
Set Types:	`set`, `frozenset`
Boolean Type:	`bool`
Binary Types:	`bytes`, `bytearray`, `memoryview`
None Type:	`NoneType`

Python does not have a character data type

Example	Data Type
x = "Hello World"	str
x = 20	int
x = 20.5	float
x = 1j	complex
x = ["apple", "banana", "cherry"]	list
x = ("apple", "banana", "cherry")	tuple
x = range(6)	range
x = {"name" : "John", "age" : 36}	dict
x = {"apple", "banana", "cherry"}	set
x = frozenset({"apple", "banana", "cherry"})	frozenset
x = True	bool
x = b"Hello"	bytes
x = bytearray(5)	bytearray
x = memoryview(bytes(5))	memoryview
x = None	NoneType

Mutable vs Immutable

Simple Definition

- **Immutable:** Cannot be changed
- **Mutable:** Can be changed

Immutable Types- int,float,string,Tuple

```
floors = 5
floors = 6

material = "Steel"
material = "Aluminium"
```

Python **creates a new value**,
doesn’t modify old one

Mutable Types: List

```
beam_lengths = [5, 6, 7]
beam_lengths[0] = 8
print(beam_lengths)

temperatures = [70, 72, 75]
temperatures.append(78)
```

Same list modified

String Formatting in Python

Why String Formatting?

If stress = 25.6789345 Mpa

Do we print:

Stress: 25.6789345 MPa

Or

Stress: 25.68 MPa

Method 1: Basic String Concatenation

```
name = "Bridge"

length = 120


print("Project Name: " + name)

print("Length: " + str(length) + " meters")
```

Output:

```
Project Name: Bridge
Length: 120 meters
```

Method 2: format() Method

```
temperature = 35

print("Temperature is {} degree Celsius".format(temperature))
```

Output:

```
Temperature is 35 degree Celsius
```

Multiple Variables

```
force = 1000
area = 50

print("Force = {} N and Area = {} sq.m".format(force, area))
```

Output:

Force = 1000 N and Area = 50 sq.m

Positional Indexing

```
print("Area = {1} and Force = {0}".format(force, area))
```

Output:

```
Area = 50 and Force = 1000
```

Decimal Control

```
stress = 25.678934

print("Stress = {:.2f} MPa".format(stress))
```

Output:

```
Stress = 25.68 MPa
```

Method 3: f-Strings (Modern & Recommended)

```
temperature = 35

print(f"Temperature is {temperature} degree Celsius")
```

Output:

```
Temperature is 35 degree Celsius
```

Expression Inside f-string

```
force = 1000

area = 50

stress = force / area


print(f"Stress = {stress} MPa")
```

Output:

```
Stress = 20.0 MPa
```

Decimal Formatting in f-string

```
force = 1000

area = 50

stress = force / area

print(f"Stress = {stress:.2f} MPa")
```

Output:

```
Stress = 20.00 MPa
```

Commonly Used String Functions

We will focus on **important and exam-relevant functions only**.

- **1. lower() and upper()**

Purpose:

Convert case.

```
material = "Steel"

print(material.lower())
print(material.upper())
```

Output:

```
steel
STEEL
```

- **2. strip(), lstrip(), rstrip()**

Removes extra spaces.

```
data = "   32 MPa   "

print(data.strip())
```

Output:

5. join()

Combine list into string.

```
values = ["32", "5", "80"]

result = ",".join(values)
print(result)
```

Output:

```
32,5,80
```

Used when writing back to file.

- **6. find()**

Find position of substring.

```
text = "Concrete Strength Test"

print(text.find("Strength"))
```

Output:

```
9
```

Returns index number.

If not found → returns `-1`

- **7. startswith() and endswith()**

Data Conditioning using Decision & Iterative Statements

- Decision Statements (if, if–else)

1. Rainfall Alert System

```
rainfall = 180

if rainfall > 150:
    print("Flood warning issued")
else:
    print("Normal rainfall")
```

2. Machine Temperature Check

```
temperature = 85

if temperature > 80:
    print("Machine overheating")
else:
    print("Temperature normal")
```

Iterative Statements (Loops)

for : Loop

- A **for loop** is used when we want to **repeat an action for each value in a given set of data**.
- Use a **for loop** when the **number of repetitions is known** or when looping through **existing data**.

Syntax of `for` Loop

for *variable* **in** *sequence* :
  statements

Part	Meaning
`for`	Loop keyword
`variable`	Temporary variable (takes one value at a time)
`in`	Connects variable to data
`sequence`	List, range, string, etc.
`:`	Start of loop block
Indentation	Mandatory (defines loop body)

Example/Problem:

1. Print rainfall recorded for 5 days.

```
rainfall = [110, 125, 140, 160, 150]

for day_rain in rainfall:
    print("Rainfall:", day_rain, "mm")
```

Output:
Rainfall: 110 mm
Rainfall: 125 mm
Rainfall: 140 mm
Rainfall: 160 mm
Rainfall: 150 mm

2. Display temperature readings of a machine.

```
temperatures = [68, 70, 75, 80, 85]

for temp in temperatures:
    print("Machine temperature:", temp, "°C")
```

Output:
Machine temperature: 68 °C
Machine temperature: 70 °C
Machine temperature: 75 °C
Machine temperature: 80 °C
Machine temperature: 85 °C

Looping Through a String

```
for x in “Looping Through":
  print(x)
```

Using `range()` with `for` Loop

Example

Floors in a Building

```
for floor in range(1, 6):
    print("Checking floor:", floor)
```

Output:

```
Checking floor: 1
Checking floor: 2
Checking floor: 3
Checking floor: 4
Checking floor: 5
```

Load Testing Cycles

```
for cycle in range(1, 4):
    print("Test cycle:", cycle)
```

Output:

```
Test cycle: 1
Test cycle: 2
Test cycle: 3
```

Using `for` Loop for Calculations

Total Rainfall

```
rainfall = [120, 130, 150]
total = 0

for r in rainfall:
    total = total + r

print("Total rainfall:", total)
```

Average Temperature

```
temperatures = [70, 75, 80]
total = 0

for t in temperatures:
    total += t

average = total / len(temperatures)
print("Average temperature:", average)
```

Common Mistakes

❌ Missing indentation

```
for r in rainfall:
print(r)
```

❌ Forgetting colon ：

```
for r in rainfall
```

❌ Using wrong variable

```
for r in rainfall:
    print(rainfall)
```

WHILE LOOP

A **`while` loop** repeats a block of code **as long as a given condition is true**.

- Keep pumping water **while** level is low
- Keep running the machine **while** temperature is safe
- Keep taking readings **while** load is within limits

Use a `while` loop when the **number of iterations is not known in advance**

Basic Syntax of `while` Loop

```
while condition:
    statements
```

Key Rules:

- Condition must be **True or False**
- Colon `:` is mandatory
- Indentation defines loop body
- Loop stops when condition becomes **False**

Problem/Example :

Print water level until it reaches safe limit (10 m).

```
water_level = 5

while water_level <= 10:
    print("Water level:", water_level, "m")
    water_level += 1
```

Output:

```
Water level: 5 m
Water level: 6 m
Water level: 7 m
Water level: 8 m
Water level: 9 m
Water level: 10 m
```

Monitor machine temperature until it exceeds 80°C

```
temperature = 70

while temperature <= 80:
    print("Machine temperature:", temperature, "°C")
    temperature += 2
```

Output:

```
Machine temperature: 70 °C
Machine temperature: 72 °C
Machine temperature: 74 °C
Machine temperature: 76 °C
Machine temperature: 78 °C
Machine temperature: 80 °C
```

`while` Loop with User Input

```
temperature = int(input("Enter temperature: "))

while temperature <= 80:
    print("Safe temperature:", temperature)
    temperature = int(input("Enter temperature: "))

print("Warning! Overheating")
```

```
Enter temperature: 45
Safe temperature: 45
Enter temperature: 60
Safe temperature: 60
Enter temperature: 69
Safe temperature: 69
Enter temperature: 70
Safe temperature: 70
Enter temperature: 80
Safe temperature: 80
Enter temperature: 88
Warning! Overheating
```

Infinite `while` Loop

```
while True:
    print("Monitoring system running")
```

Used in:

- Embedded systems
- Industrial control systems

Stopping an Infinite Loop using `break`

```
while True:
    level = int(input("Enter water level: "))
    print("Level:", level)

    if level > 10:
        print("Danger level reached")
        break
```

`while` Loop with `continue`

```
while True:
    vibration = float(input("Enter vibration value: "))

    if vibration < 0:
        continue

    print("Vibration:", vibration)
```

Global Variables

Introduction to Functions

Definition :

A function is a reusable block of code that performs a specific task.

Why Functions Are Needed

- Avoid repetition
- Improve readability
- Easy testing and modification
- Logical program structure

Syntax of a Function

```
def function_name(parameters):
    statements
    return value
```

Function Without Parameters

Site Status

```
def site_status():
    print("Construction site is active")

site_status()
```

Machine Status

```
def machine_status():
    print("Machine is operating normally")

machine_status()
```

Function With Parameters

Area of Slab

```
def slab_area(length, width):
    area = length * width
    print("Slab area:", area, "sq.m")

slab_area(12, 8)
```

Force Calculation

```
def calculate_force(mass, acceleration):
    force = mass * acceleration
    print("Force:", force, "N")

calculate_force(40, 9.8)
```

Functions with Return Values

- Use return keyword
- Store and reuse function output
- Apply functions to real calculations

Power Efficiency

```
def efficiency(output_power, input_power):
    return (output_power / input_power) * 100

eff = efficiency(450, 500)
print("Efficiency:", eff, "%")
```

OutPut:

Efficiency: 90.0 %

Concrete Volume Calculation

```
def concrete_volume(length, width, depth):
    volume = length * width * depth
    return volume

v = concrete_volume(10, 5, 0.3)
print("Concrete required:", v, "m3")
```

OutPut:

Concrete required: 15.0 m3

Multiple Return Values

```
def beam_properties(length, width):
    area = length * width
    perimeter = 2 * (length + width)
    return area, perimeter

a, p = beam_properties(5, 3)
print("Area:", a)
print("Perimeter:", p)
```

Output:

```
Area: 15
Perimeter: 16
```

Function Calling Another Function

```
def power(work, time):
    return work / time

def efficiency(work, time, input_power):
    output_power = power(work, time)
    return (output_power / input_power) * 100

print(efficiency(5000, 10, 600))
```

```
def power(work, time):
    return work / time

def efficiency(work, time, input_power):
    output_power = power(work, time)
    return (output_power / input_power)
* 100

print(efficiency(5000, 10, 600))
```

Variable Scope

Learning Objectives

- Understand local and global variables
- Identify scope-related errors
- Use global keyword correctly

What is Variable Scope?

Variable scope defines where a variable can be accessed in a program.

Local Variables

```
def load_check():
    load = 600
    print("Load inside function:", load)

load_check()
print(load)     # It will give error
```

Global Variables

```
temperature = 70

def display_temperature():
    print("Temperature:",
temperature)

display_temperature()
```

Using 'global' Keyword

```
speed = 60

def increase_speed():
    global speed
    speed += 10

increase_speed()
print("Speed:", speed)
```

Scope Rule in Python

Python follows the **LEGB Rule** for variable lookup:

Level	Meaning
L	Local (inside inner function)
E	Enclosing (outer function)
G	Global
B	Built-in

Example 1 : Local vs Enclosing Scope

```
def machine():
    temperature = 70   # Enclosing variable

    def monitor():
        temperature = 80   # Local variable
        print("Inner temperature:", temperature)

    monitor()
    print("Outer temperature:", temperature)

machine()
```

Output:

Inner temperature: 80
Outer temperature: 70

Example 2: Accessing Outer Variable (Enclosing Scope)

Load Calculation

```
def structure():
    load = 500   # Enclosing variable

    def check_load():
        print("Load:", load, "kN")

    check_load()

structure()
```

Output:
Load: 500 kN

Explanation:

- `load` is **not local** to `check_load()`
- Python finds it in **enclosing scope**
- Allowed ✔️

Example 3: Modifying Enclosing Variable (Error Case)

```
def machine():
    temperature = 70

    def monitor():
        temperature = temperature + 5   # ERROR
        print(temperature)

    monitor()

machine()
```

Example 4: Using `nonlocal` Keyword (Correct Way)

```
def machine():
    temperature = 70

    def monitor():
        nonlocal temperature
        temperature += 5
        print("Updated temperature:", temperature)

    monitor()
    print("Final temperature:", temperature)

machine()
```

O/P:

Updated temperature: 75
Final temperature: 75

What is a Lambda (Anonymous) Function?

Definition

A lambda function is a small anonymous function defined without a name, used for short operations.

Syntax of Lambda Function

lambda arguments: expression

Example:

lambda x: x * x

- `lambda` → keyword
- `x` → input
- `x * x` → expression (no `return`)
- ⚠️ Lambda functions **automatically return** the result.

Lambda vs Normal Function

Normal Function	***Lambda Function***
Has name	No name
Multiple lines	Single line
Uses `return`	No `return`
Complex logic	Simple logic

```python
x = lambda a : a + 10
print(x(5))
```

OutPut: 15

```python
x = lambda a, b : a * b
print(x(5, 6))
```

OutPut: 30

```
def design_check(limit):
    return lambda load: load <= limit

check = design_check(500)
print(check(450))  # True
print(check(600))  # False
```

Introduction to Modules

- Understand what a module is
- Create and use their own module
- Use different import statements

Problem without modules:

- Large programs
- Repeated code
- Difficult maintenance

What is a Module?

A **module** is a Python file containing functions, variables, and classes that can be reused in other programs.

Every ***`.py`*** *file is a module.*

Creating a User-Defined Module

Step 1: Create a module file

File name:
`engineering_calculations.py`

```
def area_rectangle(length, width):
    return length * width

def stress(force, area):
    return force / area

material = "Steel"
```

Step 2: Use the module in another file

File name: *main.py*

```
import engineering_calculations

print(engineering_calculations.area_rectangle(10, 5))
print(engineering_calculations.stress(1000, 50))
print(engineering_calculations.material)
```

Different Ways to Import Modules

1 Import full module

```
import math
print(math.sqrt(25))
```

2 Import with alias

```
import math as m
print(m.pi)
```

Packages & Standard Library Modules

- Understand packages
- Use Python standard library modules
- Apply modules to engineering problems

What is a Package?

A **package** is a collection of related modules organized in folders.

Package Structure

```
Engineering (Folder)
│
├── civil.py
├── mechanical.py
└── __init__.py
```

Example Module: `civil.py`

```
def concrete_volume(l, w, d):
    return l * w * d
```

Example Module: `mechanical.py`

```
def power(work, time):
    return work / time
```

Using the Package

```
from engineering import civil, mechanical


print(civil.concrete_volume(10, 5, 0.3))
print(mechanical.power(5000, 10))
```

Python Standard Library

The **standard library** is a collection of built-in modules that come with Python.

Common Standard Library Modules

`math` Module

```
import math
print(math.sqrt(49))
print(math.pi)
```

Uses:

- Geometry
- Trigonometry
- Structural calculations

`statistics` Module

```
import statistics
data = [10, 12, 15, 20]
print(statistics.mean(data))
```

Data Handling using Files

Why File Handling?

What happens to data when the program stops?

- Variables → Temporary storage
- Files → Permanent storage

Types of Files

a) Text Files

- Human readable
- `.txt`, `.csv`

File Handling

The key function for working with files in Python is the `open()` function and `close()` function.

Syntax:

```
file = open("filename", "mode")

# operations

file.close()
```

In addition you can specify if the file should be handled as binary or text mode

`"t"` - Text - Default value. Text mode
`"b"` - Binary - Binary mode (e.g. images)

File Modes in Python

There are four different methods (modes) for opening a file:

- "r" **- Read** - Default value. Opens a file for reading, error if the file does not exist
- "a" **- Append** - Opens a file for appending, creates the file if it does not exist
- "w" **- Write** - Opens a file for writing(**overwrites existing data**), creates the file if it does not exist
- "x" **- Create** - Creates the specified file, returns an error if the file exists

```
f = open("demofile.txt")
```

```
f = open("demofile.txt", "rt")
```

Writing Data to a File

Example 1: Weather Data

```
file = open("weather.txt", "w")

file.write("Day 1 Temperature: 32\n")

file.write("Day 2 Temperature: 34\n")

file.close()
```

Reading Data from a File

Read Entire File

```
file = open("weather.txt", "r")

data = file.read()

print(data)

file.close()
```

Read Line by Line

```
file = open("weather.txt", "r")

for line in file:                    #File object is iterable
    print(line.strip())              #strip() removes extra spaces/ newlines

file.close()
```

Appending Data to a File

```
file = open("machine_log.txt", "a")

file.write("Operating Temp: 85°C\n")

file.close()
```

`with` Statement

Why `with`?

Automatically closes file

```
with open("weather.txt", "r") as file:
    for line in file:
        print(line)
```

You should always close your files. In some cases, due to buffering, changes made to a file may not show until you close the file.

```python
# Weather Data Management using File Handling

FILENAME = "weather_log.txt"

def write_data():
    """Write fresh weather data to file"""
    with open(FILENAME, "w") as file:
        file.write("Day,Temperature,Rainfall\n")
        file.write("1,32,5\n")
        file.write("2,34,0\n")
    print("Weather data written successfully.\n")

def append_data():
    """Append new day weather data"""
    day = int(input("Enter day number: "))
    temp = float(input("Enter temperature (°C): "))
    rain = float(input("Enter rainfall (mm): "))

    with open(FILENAME, "a") as file:
        file.write(f"{day},{temp},{rain}\n")

    print("New weather data appended.\n")

def read_data():
    """Read and display weather data"""
    try:
        with open(FILENAME, "r") as file:
            print("\n--- Weather Records ---")
```